EVENTO 2026 · PALESTRA

IA generativa para desenvolvedores

Do chat que sugere código ao agente que trabalha no seu repositório.

Rodrigo

Python ChatGPT LangChain LangGraph Langflow n8n MCP Claude Code

O QUE MUDOU

O desenvolvedor passou de autor a revisor

ANTES

Você escreve, a IA completa a linha

```
def total(itens):
    return sum(i.preco for i in itens)
```

Contexto: o arquivo aberto.

AGORA

A IA executa, você define e revisa

- ✓ lê o projeto
- ✓ executa comandos
- ✓ testa
- ✓ entrega um diff

Contexto: o repositório, as ferramentas e as regras que você define.

Como chegamos aqui em quatro anos?

2022

2022 · 2023 · 2024 · 2025 · 2026

O chat chega ao cotidiano

ChatGPT — 30 de novembro

A interface de conversa chega ao cotidiano.

conversa

```
> como leio um CSV em Python?
Use o pandas:
pd.read_csv("dados.csv")
```

LangChain — outubro · 1º pacote Python

O desenvolvedor conecta o modelo à aplicação.

Aplicação — seu código e suas regras → **Modelo** — uma chamada por API → **Dados** — documentos e ferramentas

Marcos de acesso e composição, não o início da pesquisa em IA. Fontes: anúncio da OpenAI (30/11/2022); histórico de versões do LangChain.

2023

2022 — 2023 — 2024 — 2025 — 2026

Frameworks, fluxos e os primeiros agentes

Construir um fluxo

LangChain · Langflow · n8n

Você desenha o caminho; o modelo é uma etapa.



Experimentar agentes

AutoGPT

O agente escolhe a próxima ação até cumprir o objetivo.



> O modelo participa da escolha das ações e recebe os resultados das ferramentas.

Fontes: repositórios oficiais do LangChain, Langflow e AutoGPT (2023).

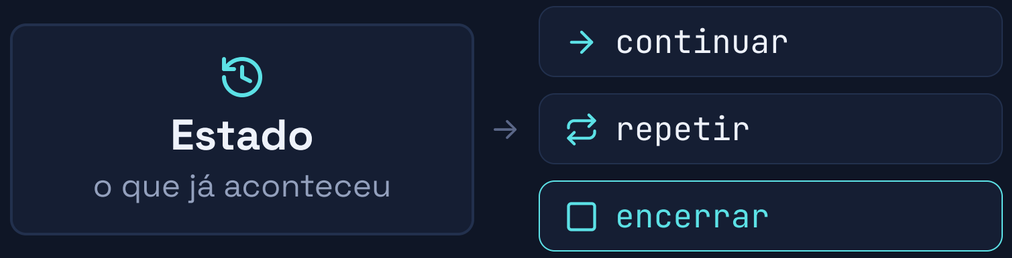
2024

2022 · 2023 · **2024** · 2025 · 2026

Estado, ciclos e um protocolo para ferramentas

LangGraph `janeiro`

O fluxo guarda o que já aconteceu e decide continuar, repetir ou encerrar.



MCP · Model Context Protocol

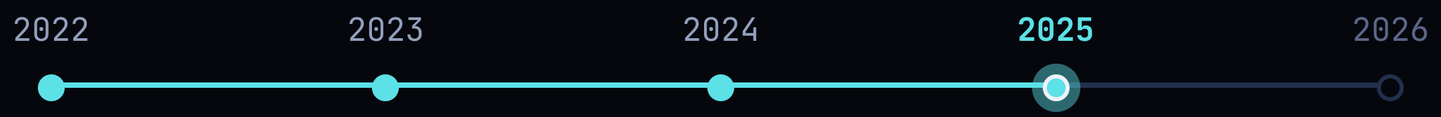
`25 de novembro`

Um protocolo aberto para conectar ferramentas e dados.



O MCP padroniza a integração; as permissões continuam na implementação. Fontes: LangGraph (jan/2024); anúncio da Anthropic (25/11/2024) e modelcontextprotocol.io.

2025



A IA passa a trabalhar dentro do projeto

meu-projeto · terminal

```
$ claude "corrija o filtro de busca"
lendo src/busca.py
- if filtro and termo:
+ if filtro is not None and termo:
$ pytest -q
12 passed
diff pronto para revisão
```

Claude Code

preview em fevereiro · disponível em maio

Codex CLI

abril · terminal

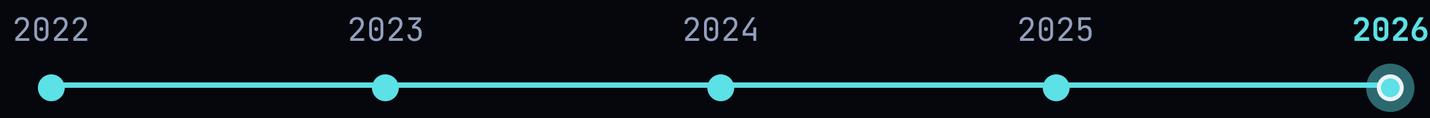
Agent Skills

outubro

Procedimentos, scripts e referências carregados conforme a tarefa.

Representação didática. A CLI conduz a execução; o modelo é uma escolha separada. Fontes: anúncios da Anthropic (fev, mai e out/2025) e da OpenAI (abr/2025).

2026



Agentes que trabalham enquanto você revisa

Auto mode

Sem pedir aprovação a cada comando; um classificador revisa o que for arriscado antes de executar.

Subagentes

Trabalho independente em paralelo, sem poluir a conversa principal.

Sessões em paralelo

`claude -w feature-x` abre uma sessão em outra worktree. Também no desktop, na web e em agendamentos.

Pedido → **Agente** → **Checagem automática** → **Revisão humana**

Recursos conforme a documentação oficial (code.claude.com/docs) em novembro de 2026. Autonomia sempre dentro de permissões.

ONDE USAR

Três lugares para trabalhar com um agente de código



IDE

Um editor feito para agentes

Cursor
Antigravity
Kiro



Plugin

Um agente dentro do seu editor

GitHub Copilot
Extensão do Claude Code para VS Code e JetBrains



CLI

Um agente no seu terminal

Claude Code
Codex CLI
Gemini CLI
Cursor CLI
OpenCode · Pi

também no desktop, na web e no VS Code

> O tipo de ferramenta é o mesmo; muda onde você conversa com ela. Nesta palestra, vamos de terminal.

Lista ilustrativa, não exaustiva. As ferramentas mudam rápido: nomes conferidos nas páginas oficiais antes do evento.

MODELO MENTAL

Como o Claude Code trabalha: um loop com ferramentas



Você pode interromper a qualquer momento com `Esc`; o trabalho feito até ali fica.

Diagrama simplificado, baseado em "How Claude Code works" (code.claude.com/docs).

PRIMEIROS PASSOS

Começar leva três comandos

terminal

```
$ curl -fsSL https://claude.ai/install.sh | bash
$ cd meu-projeto && claude
> /init
analisando o projeto…
✓ CLAUDE.md criado
```

CLAUDE.md

O `/init` gera um rascunho. O trabalho depois é **cortar** linhas: a doc recomenda menos de 200.

OUTRAS FORMAS

macOS · Homebrew

```
brew install --cask claude-code
```

Windows · PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

Também

VS Code, JetBrains, desktop e web

Comandos conferidos no Quickstart de code.claude.com/docs em novembro de 2026. Saída do terminal simplificada.

DIA A DIA

Os comandos que você usa todo dia

Sessão

`/`	lista comandos e skills
`/help`	ajuda e comandos disponíveis
`@arquivo`	aponta um arquivo no pedido
`Shift+Tab`	alterna o modo de permissão
`/model` `/effort`	troca o modelo e o esforço

Contexto

`/context`	o que está ocupando o contexto
`/clear`	conversa nova entre tarefas diferentes
`/compact <foco>`	resume mantendo o importante
`/btw`	pergunta rápida que não entra no histórico

Controle

`Esc`	para e redireciona; o feito fica
`Esc Esc` `/rewind`	volta código e conversa a um ponto anterior
`/permissions`	regras allow, ask e deny
`/hooks`	mostra os hooks configurados

Retomar e automatizar

`claude -c`	continua a última conversa
`claude -r <nome>`	retoma uma sessão (`--resume`)
`/rename`	dá nome à sessão
`claude -p "..."`	uma execução, para scripts e CI
`/code-review`	revisa o diff atual

Lista resumida, conferida em novembro de 2026. Referência completa: code.claude.com/docs/en/commands

CONTEXTO

Diga ao Claude o que ele não vê no código

`toda sessão`

CLAUDE.md

Comandos de build e teste, convenções, armadilhas. Curto.

`por pasta`

`.claude/rules/` com `paths:`

Só carrega quando ele mexe naquela parte do código.

`sob demanda`

Skills

Procedimentos em `.claude/skills/`, também como `/nome`.

`integrações`

MCP

`claude mcp add` para sistemas sem CLI: Notion, banco, issues.

CLAUDE.md

```
# Commands
npm test · npm run typecheck
# Conventions
named exports; zod nas rotas
# Gotchas
migrations/ é gerado: não editar
```

Rode `/context` para confirmar o que foi carregado.

Exemplo de CLAUDE.md enxuto, ilustrativo. Camadas conforme "Memory" e "Best practices" (code.claude.com/docs).